

Sistemas Embebidos
Trabajo práctico N°2 - Año 2025
Sistemas Operativos para sistemas embebidos - FreeRTOS

Objetivos

- Utilizar FreeRTOS
- Repasar conceptos de sistemas operativos.
- Implementar un sistema de IoT completo.

Metodología

Trabajo individual o grupal. 2 estudiantes por grupo máximo.

Tiempo de realización estimado: 2 a 3 clases.

Aprobación

- Mostrar en clases la aplicación funcionando correctamente.
- Enviar los programas de computación implementados a través de la plataforma Moodle.
- Escribir un breve informe y enviarlo a través de la plataforma Moodle.

Materiales necesarios

- Placas Arduino UNO y circuito con leds sensores (provistas por la cátedra).
- Entorno de desarrollo de Arduino UNO (puede descargarse de <https://www.arduino.cc/en/software>).
- Intérprete de lenguaje de programación Python 3.
- Módulo pySerial para la comunicación serial (ver apunte sobre PySerial en aula abierta).
- Servidor web. Puede utilizar:
 - Frameworks de desarrollo web. Se sugiere Flask, FastAPI o Django (en el aula abierta encontrará un apunte sobre Flask).
 - Apache Web Server (En el trabajo práctico N°6 de la asignatura Redes de Computadoras se dan instrucciones de instalación y uso).
- Librerías para Arduino IDE FreeRTOS de Richard Barry. Puede agregarla en Arduino IDE desde el menú "Tools->Manage Libraries" y buscar la librería "FreeRTOS", verificar que diga "implementado para AVR (Uno, Nano, Leonardo, Mega)".
- Opcional: Puede escribir el código a ejecutar en el Arduino UNO empleando un simulador adecuado. La cátedra provee una simulación con el circuito listo para ser utilizado en el enlace: <https://wokwi.com/projects/391281841803809793>
El simulador permite instalar las librerías de FreeRTOS.
Luego podrá copiar el código a la memoria flash del Arduino UNO.

Chatbots de IA sugeridos:

Si bien todos los chatbot de IA son adecuados como soporte para resolver este trabajo práctico, la cátedra ha verificado que las siguientes son adecuadas:

- ChatGPT de OpenAI (<https://chatgpt.com/>). Funciona muy bien con FreeRTOS. Detecta errores puntuales.
- Gemini de Google (<https://gemini.google.com/>). Comprende FreeRTOS y brinda ayuda simple de entender si se le pide que explique conceptos o cree ejemplos, pero comete algunos errores y fallas.

Actividad 1:

1. Instalar la librería para Arduino UNO de FreeRTOS y analizar los ejemplos. Se sugiere analizar con detenimiento y en el orden indicado los siguientes ejemplos (encontrará los ejemplos en Edit->Examples->FreeRTOS):
 - a. Blink_AnalogRead: ejemplo más básico.
 - b. AnalogRead_DigitalRead: para comprender el uso de semáforos.
 - c. ArrayQueue: para comprender el uso de colas.
 - d. Mutex: para comprender el uso de mutex.
 - e. Interrupts: para comprender el uso de semáforos dentro de rutinas de servicios de interrupciones.

En el caso de usar el simulador, puede instalar Arduino IDE (<https://www.arduino.cc/en/software>) para copiar el código de los ejemplos al simulador.

2. Implementar una aplicación que realice las siguientes tareas usando FreeRTOS:
 - a. Lea constantemente el valor de la intensidad luminosa.
 - b. Cada 3 segundos, envíe a través del puerto serial el último valor leído (Nota: la escritura en el monitor serial demora cierto tiempo. No debe ser interrumpida por otra tarea). Muestre el valor leído en una página web y en la aplicación Python.
 - c. La lectura puede iniciarse y detenerse a través de los pulsadores de la placa Arduino y desde botones en una página web.
 - d. Parpadee el led 11 cada un segundo si la lectura está activada. No parpadee el led 11 si la lectura está desactivada.
 - e. Si detecta que el valor de intensidad luminosa supera 800, encienda una alarma que:
 - i. Haga parpadear el led 12 con periodo de 0.1 segundo
 - ii. Indique en la aplicación web la situación.
 - iii. La alarma se desactiva solo si la lectura se desactiva (la alarma no debe desactivarse si la lectura vuelve a estar por debajo de 800).

Sugerencias de implementación:

- Cada ítem puede ser configurado como una tarea de FreeRTOS.
- El manejo de pulsadores debería realizarse mediante interrupciones. Sin embargo, la rutina de servicio de la interrupción debería ejecutar un código de mínima duración, como habilitar una tarea, siendo esta tarea la que active o desactive el sistema.
- Deberá asignar memoria para la pila de las tareas. Debe tener en cuenta que la memoria del Arduino Uno es limitada. Asignar poca memoria puede hacer que la tarea no funcione correctamente. Asignar demasiada memoria puede hacer que se agote la memoria del Arduino Uno, y el programa completo no funcione (ver Anexo 2).
- Las variables de tipo `string` hacen uso ineficiente de la memoria, por no ser variables de tamaño fijo. Su uso inadecuado puede llevar a sobrescribir otras variables e incluso código. Se sugiere utilizar strings de tamaño fijo y el menor tamaño posible. Una mejor solución puede ser el uso de array de caracteres, ya que los mismos hacen uso de una cantidad fija de memoria, aunque su manejo puede ser un poco más complejo.

Anexo 1: Librerías necesarias

En <https://www.freertos.org/a00106.html> se listan todas las primitivas de FreeRTOS y las librerías que necesita para cada función.

Manejo de interrupciones: Revise la función `attachInterrupt` en <https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/>

Anexo 2: Tamaño de pila

La función:

```
UBaseType_t uxTaskGetStackHighWaterMark( TaskHandle_t xTask );
```

Retorna la porción de la pila no utilizada por la tarea indicada. Se retorna en cantidad de palabras (16 bits en Arduino). Permite saber si se puede disminuir el tamaño de la pila de una tarea.

La función indicada consume memoria en la pila, por lo que debe ser utilizada con precaución.